

Quarry: Technical Architecture Specification

Local Semantic Search for AI Agents

July 2026

Contents

1	Overview	3
1.1	Design Principles	3
2	System Architecture	3
2.1	Deployment Topology	3
2.2	Surface Summary	4
3	Module Responsibilities	4
3.1	Data Layer (<code>src/quarry/db/</code>)	4
3.2	Ingestion Pipeline (<code>src/quarry/ingestion/</code>)	4
3.3	Retrieval Seam (<code>src/quarry/retrieval/</code>)	5
3.4	Format Processors (<code>src/quarry/extractors/</code>)	5
3.5	Backends	6
3.6	Capture and Redaction	6
3.7	Fetch Safety (SSRF)	6
3.8	Sync and Registry	7
3.9	Always-on Watch/Index Loop (DES-045)	7
3.10	Surfaces	9
3.11	Support	10
4	Daemon Model	11
4.1	Why a Daemon	11
4.2	Process Boundary and Loopback Authentication (v2.2)	11
4.3	Client/Engine Boundary (I1)	11
4.4	MCP Transport (<code>quarry mcp</code>)	12
4.5	CLI and the Daemon	12
5	MCP Tool Surface	12
6	HTTP Server	12
6.1	ASGI Application	12
6.2	Endpoints	13
6.3	Port File	13
7	Configuration	13
7.1	Settings	13
7.2	Named Databases	14
7.3	Chunking Tuning	15
8	Search and Retrieval	15
8.1	Embedding Model	15
8.2	OCR	15
8.3	Search Quality Tips	15
8.4	Hybrid Search	16
8.5	Retrieval Seam and Evaluation	16
8.6	Ethos Extension Integration	16

9	Logging and Exceptions	17
9.1	Logging Standard	17
9.2	Exception Handling	17
10	Security	18
10.1	Bearer Token Authentication	18
10.2	Request Admission (CSRF)	18
10.3	CORS	18
10.4	TLS and TOFU Certificate Pinning	18
10.5	Log Safety (CWE-532, CWE-117)	19
10.6	Capture PII Redaction (DES-036)	19
11	Deployment	19
11.1	Single Machine	19
11.2	Network Mode (Remote GPU)	20
11.3	Client (Remote Access)	20
11.4	Daemon (<code>quarryd</code>)	20
11.5	Container (Fly.io)	20
12	Performance	20
12.1	Critical Path	20
12.2	Fire-and-Forget Writes	21
12.3	Single Table Design	21
13	Plugin and Hook System	21
13.1	Session Start	21
13.2	Web Fetch Capture	21
13.3	Pre-Compact Capture	21
14	CLI Commands	22
15	Test Architecture	22
16	Formal Model	23
A	Operation Concurrency Model	24
A.1	Concurrency Matrix	24
A.2	Design Rationale	24
A.3	Task Lifecycle	25

1 Overview

Quarry is a local semantic search engine. It ingests documents in 20+ formats, embeds them with a local ONNX model, stores vectors in LanceDB, and serves semantic search to AI agents (Claude Code, Claude Desktop) and a macOS menu bar app.

The system is **daemon-first** (DES-031 v2.2, ACCEPTED and complete): a single **quarryd** process holds the engine, and the CLI, MCP, and library are pure clients that reach it over a versioned wire protocol — the daemon is the intermediary service layer. The engine runs only in **quarryd** (the **quarry serve** subcommand is retired); loopback requests are authenticated with a mode-0600 **serve.token** resolved through **ClientConfig**; and every client surface — the CLI, the **quarry mcp** stdio server, hooks, and the library — reaches the daemon through one **QuarryClient** over the **/v1** REST contract. No client process imports or constructs the engine; an import-linter contract and a runtime engine-sabotage test enforce that boundary structurally in CI (the epic **quarry-ma6f/quarry-ynvs** is closed). The always-on watch/index loop that daemon-first exists to enable (DES-045) keeps every registered directory’s index fresh with no human action.

1.1 Design Principles

- P1. Daemon-first.** The **quarryd** process is the single engine (embedding model, LanceDB, pipeline, sync registry); the CLI, MCP, and library are clients over a versioned REST contract (DES-031 v2.2, ACCEPTED and complete). The prior *library-first* principle — the core library did all work and the surfaces were thin wrappers — described the as-built state DES-031 v2 replaced.
- P2. Local by default.** Everything runs on the user’s machine. No API keys, no cloud accounts. The embedding model downloads once (~120 MB, int8 quantized).
- P3. One daemon, many clients.** A single **quarryd** process loads the embedding model once and serves all sessions over the REST wire; supervised by launchd **KeepAlive** / systemd **Restart=always**.
- P4. Fire-and-forget I/O.** Side-effect MCP tools return immediately and process in a bounded thread pool. Search is synchronous.
- P5. Named databases.** Each database is fully isolated: its own LanceDB directory, sync registry, and vector index. Work/personal separation without configuration files.

2 System Architecture

2.1 Deployment Topology

Single machine (localhost TLS):

```
          stdio HTTPS /v1 (TLS)
Claude Code <-----> quarry mcp <-----> quarryd
          MCP JSON-RPC (QuarryClient) pinned CA cert (one daemon)
```

Remote server (GPU):

```
Mac (client) Linux (server)
          stdio HTTPS /v1 (TLS)
Claude Code <-----> quarry mcp <-----> quarryd
          MCP JSON-RPC (QuarryClient) pinned CA cert (GPU daemon)

CLI -----> HTTPS GET/POST /v1 -----> quarryd
          Bearer + pinned CA
```

Quarry’s own Claude Code plugin runs **quarry mcp** directly (a FastMCP stdio server whose tools are **QuarryClient** calls over the daemon’s **/v1** REST API); it no longer routes through **mcp-proxy** (DES-031 v2.2). Because a single **quarryd** holds the resident engine, every Claude Code tab shares one warm model rather than spawning a Python process that loads ~200 MB of its own. Installed services use TLS with TOFU certificate pinning (DES-020) — even on localhost. (**mcp-proxy** is an MCP stdio↔daemon transport, not a REST-API client; **quarry install** still fetches it because it remains a

supported transport for other MCP clients, but it is no longer on quarry’s own MCP path — the plugin runs `quarry mcp` directly.)

When a remote server is configured (DES-021), the CLI routes all data commands over HTTPS to the server, with only authentication and administration commands remaining local. `find`, `show`, `status`, `ingest`, `remember`, `delete`, `sync`, all `list` subcommands, and directory `register/deregister` route remotely. `use` is local by design: it sets the client’s persistent default database, but the remote server is fixed to the database it was started with, so switching databases is a client-side setting rather than a remote call.

2.2 Surface Summary

Surface	Transport	Purpose
CLI	QuarryClient over HTTPS /v1	Interactive use from terminal
MCP server	stdio (<code>quarry mcp</code> , a <code>QuarryClient</code>)	Claude Code/Desktop tools
HTTP server	REST JSON /v1 (FastAPI in <code>quarryd</code>)	CLI, MCP, menu bar app
Plugin	Hook shell commands	Auto-register, auto-ingest, compaction safety

3 Module Responsibilities

3.1 Data Layer (`src/quarry/db/`)

The database facade composes five subsystems behind a single entry point (`Database`), so callers pass one object rather than a raw LanceDB connection.

Module	File	Responsibility
Facade	<code>src/quarry/db/facade.py</code>	<code>Database</code> — single entry point composing the store, search, catalog, schema, and optimizer subsystems (DES-031).
Chunk store	<code>src/quarry/db/chunk_store.py</code>	Insert, progressive-flush, delete, and count operations on the <code>chunks</code> table.
Chunk search	<code>src/quarry/db/chunk_search.py</code>	Vector-similarity search under the cosine metric.
Chunk catalog	<code>src/quarry/db/chunk_catalog.py</code>	List documents, list collections, and fetch page text.
Optimizer	<code>src/quarry/db/optimizer.py</code>	Table compaction, FTS index rebuild, and per-collection indexing.
Schema	<code>src/quarry/db/schema.py</code>	Chunks-table schema and idempotent migration (agent-memory columns, FTS index).
Storage	<code>src/quarry/db/storage.py</code>	LanceDB connection, database discovery, and size formatting.

Supporting types: `src/quarry/models.py` (immutable `PageContent`, `Chunk`, `PageAnalysis` data-classes), `src/quarry/types.py` (structural Protocols for LanceDB, OCR, and embedding backends), `src/quarry/results.py` (`SearchResult`, `SearchFilter`, `WORST.CASE.DISTANCE`), and `src/quarry/config.py` (`Settings`, `resolve_db_paths()`, ONNX model constants).

3.2 Ingestion Pipeline (`src/quarry/ingestion/`)

Module	File	Responsibility
Pipeline	src/quarry/ingestion/pipeline.py	Orchestrator: dispatch by format, chunk, embed, store. Entry points <code>ingest_document</code> , <code>ingest_content</code> , <code>ingest_url</code> .
Chunker	src/quarry/ingestion/chunker.py	Split page text into overlapping chunks; respects page and heading boundaries.
URL ingester	src/quarry/ingestion/url_ingester.py	Fetch, parse HTML, chunk, embed, store. <code>ingest_url</code> takes an opt-in <code>content_scrubber</code> (see PII redaction).
Web fetch	src/quarry/ingestion/web_fetch.py	Fetch a URL over HTTP(S) and return validated HTML text.
Ingest stats	src/quarry/ingestion/ingest_stats.py	Per-format ingest counters merged into an ingest result.

Format extraction lives in `src/quarry/extractors/`; embedding and OCR provider selection in `src/quarry/ingestion/provider.py`, `src/quarry/ingestion/backends.py`, and `src/quarry/ingestion/ocr_local`.

3.3 Retrieval Seam (src/quarry/retrieval/)

Hybrid search runs through one parameterizable seam shared by every surface and by the eval harness, so the production path and the committed baseline cannot drift.

Module	File	Responsibility
Service	src/quarry/retrieval/service.py	SearchService — the single call every surface (CLI, HTTP, MCP) makes for search. Kills remote/local divergence (bug class 3).
Hybrid	src/quarry/retrieval/hybrid.py	HybridRetriever — vector + BM25 channels fused with RRF, behind a config.
Config	src/quarry/retrieval/config.py	RetrievalConfig — frozen retrieval knobs (RRF k , over-fetch, cosine metric, decay rate, reranker). Defaults reproduce shipped search bit-for-bit.
Protocols	src/quarry/retrieval/protocols.py	Retriever and Reranker structural interfaces (one method each).
Fusion	src/quarry/retrieval/fusion.py	RrfFusion — Reciprocal Rank Fusion with optional temporal decay.
Reranker	src/quarry/retrieval/reranker.py	NullReranker — pass-through Null Object when reranking is off.

3.4 Format Processors (src/quarry/extractors/)

Each extractor implements a common Protocol (`src/quarry/extractors/protocol.py`) and converts a source format into `list[PageContent]`:

Module	File	Formats
PDF	src/quarry/extractors/pdf_extractor.py	PDF (text via PyMuPDF, OCR fallback for image pages)
Image	src/quarry/extractors/image_extractor.py	PNG, JPG, TIFF, BMP, WebP (OCR)
Text	src/quarry/extractors/text_extractor.py	TXT, MD, LaTeX, DOCX
Code	src/quarry/extractors/code_extractor.py	30+ languages (tree-sitter section splitting)
HTML	src/quarry/extractors/html_extractor.py	HTML (boilerplate stripping, Markdown conversion)
Spreadsheet	src/quarry/extractors/spreadsheet_extractor.py	XLSX, CSV (LaTeX tabular serialization)
Presentation	src/quarry/extractors/presentation_extractor.py	PPTX (slide-per-chunk with notes)

3.5 Backends

Module	File	Responsibility
Factory	src/quarry/ingestion/backends.py	Thread-safe factory with double-checked locking. <code>get_ocr_backend()</code> and <code>get_embedding_backend()</code> return cached singletons.
OCR	src/quarry/ingestion/ocr_local.py	Local OCR via RapidOCR (offline, no setup).
Embeddings	src/quarry/embeddings.py	Local ONNX embedding (snowflake-arctic-embed-m-v1.5, 768-dim, 512 tokens). Vectors are L2-normalized to unit length.
Provider	src/quarry/ingestion/provider.py	ONNX Runtime execution provider auto-detection (CUDA/FP16 or CPU/int8).

3.6 Capture and Redaction

Session transcripts and WebFetch auto-captures are scrubbed at write time before any bytes reach a git-tracked (and soon pushed) surface (DES-036).

Module	File	Responsibility
Capture writer	src/quarry/capture.py	CaptureWriter — the single choke point both <code>.md</code> producers route through. Scrubs, then atomically writes.
Scrubber	src/quarry/scrub.py	Scrubber — ordered redaction passes (secrets, paths, emails, hostname, profanity). Idempotent.
Scrub rules	src/quarry/scrub_rules.py	Secret-detection regex catalog applied by the scrubber.
Capture URL	src/quarry/capture_url.py	CaptureUrl — strips user-info/query/fragment from a captured URL's metadata form.
Web capture	src/quarry/web_capture.py	Parse a <code>PostToolUse</code> WebFetch payload into URL + fetched content.
Transcript	src/quarry/transcript.py	The session transcript file and its derived identifiers.
Backfill	src/quarry/backfill.py	Backfill historical session transcripts through the capture writer.

3.7 Fetch Safety (SSRF)

Every outbound fetch — the leaf-page fetch and the sitemap crawl — runs the address-safety gate (`UrlSafetyCheck`) against the *resolved* address on *every* hop, fail-closed, before the socket opens (DES-043). The route boundary validates only the initial caller-supplied URL; the gap this closes is that redirect targets and sitemap-discovered URLs never reached that gate — an authenticated caller could otherwise reach 169.254.169.254, loopback, or a private range via a redirect from a public server or an internal URL listed in a sitemap.

Module	File	Responsibility
URL safety	<code>src/quarry/url_safety.py</code>	<code>UrlSafetyCheck</code> — classifies a URL’s resolved address (rejects link-local, loopback, RFC-1918, CGNAT, unspecified, IPv4-mapped-IPv6, and the NAT64 well-known prefix; checks all <code>getaddrinfo</code> records). In core, so the daemon route and the fetch layer share one classifier.
Redirect guard	<code>src/quarry/ingestion/ssrf_redirect.py</code>	<code>SsrfGuardedRedirectHandler</code> plus the shared <code>GUARDED_OPENER</code> : re-gates every 30x <code>Location</code> before it is opened.
Sitemap client	<code>src/quarry/sitemap_web_client.py</code>	<code>GatedSitemapWebClient</code> — USP’s <code>AbstractWebClient</code> , gating every sitemap-index, <code>robots.txt</code> , and sub-sitemap fetch at every recursion depth (a block is non-retryable, so USP never connects).
Pinned connection	<code>src/quarry/ingestion/pinned_connection.py</code>	<code>PinnedHTTP(S)Connection</code> — performs one <code>getaddrinfo</code> , validates every record, and connects the socket to a validated IP literal from that same set; no second resolution exists, so the gate-vs-connect DNS-rebind window is eliminated by construction (DES-044). Installed into the shared <code>GUARDED_OPENER</code> , so every fetch path and redirect hop inherits the pin.

A single `getaddrinfo` per connection eliminates the gate-vs-socket DNS-rebind TOCTOU by construction: the socket connects to an address the gate has just validated, never an independent resolution (DES-044, closing quarry-kmzo/quarry-ljym). The pin narrows the *address*, not the *trust* — `self.host` is never mutated, so SNI, the `Host` header, and certificate verification stay on the hostname (never the pinned IP), and the public-fetch context keeps the system trust store, kept separate from the daemon-RPC pinned-CA context.

3.8 Sync and Registry

Module	File	Responsibility
Sync	<code>src/quarry/sync.py</code>	Directory sync: discover files, compute delta (new/changed/deleted), ingest/delete accordingly.
Discovery	<code>src/quarry/sync_discovery.py</code>	Walk, filter, and hash files for the sync delta.
Ingest	<code>src/quarry/sync_ingest.py</code>	Progressive per-collection ingest (producer/consumer with within-file resume).
Registry	<code>src/quarry/sync_registry.py</code>	SQLite registry of watched directories and per-file state.

3.9 Always-on Watch/Index Loop (DES-045)

Continuous incremental indexing — the first-class daemon-first rationale (DES-031) — is a daemon-owned `src/quarry/daemon/watch_loop.py` `WatchLoop` that keeps every registered directory’s index fresh with no human action. It is a *producer only*: it watches the filesystem, debounces edit bursts, and submits index work to the existing DES-042 queue, writing no LanceDB table directly. So it inherits the whole serialization stack unchanged — DES-042 per-table FIFO → DES-034 `ProgressiveIndexer/_write_lock` → DES-026 WAL — and the queue’s routing key becomes `(database, collection)`, extending single-writer-per-table across the roster. Three producers (initial scan on start, the live watch, an explicit `quarry sync`) all feed the one queue and thus cannot race.

Module	File	Responsibility
Watch loop	<code>src/quarry/daemon/watch_loop.py</code>	The <code>WatchLoop</code> : per-database watch sets + observer lifecycle, initial scan, start/stop watching, <code>request_scan</code> . Producer into the queue.
Reconcile	<code>src/quarry/daemon/watch_reconcile.py</code>	The <code>watch_safety_scan_s</code> roster reconcile: re-enumerate, start watching new databases, re-submit shed scans, run the orphan sweep (DES-045a).
Submit	<code>src/quarry/daemon/watch_submit.py</code>	Turns a debounced delta into per-file or bulk <code>IngestUnits</code> and enqueues them.
Roster	<code>src/quarry/daemon/watch_roster.py</code>	Per-database connections + watch sets enumerated from the on-disk roster (trust boundary: only roster DBs are opened).
Scratch guard	<code>src/quarry/scratch_paths.py</code>	<code>ScratchGuard</code> : refuses a temp/scratch/VCS root and skips git-ignored paths at every producer seam (DES-045b).
Removal	<code>src/quarry/daemon/registration_lifecycle.py</code>	Deregister / subsume / retain: marks a collection in <code>pending_purge_collections</code> on non-keep-data removal; records retained-collection identity (DES-045a).
Purge	<code>src/quarry/daemon/purge_service.py</code>	The orphan sweep + <code>CollectionPurgeJob</code> : deletes only marked collections still holding chunks, re-checked under its own transaction.
Route key	<code>src/quarry/daemon/route_key.py</code>	<code>RouteKey = (database, collection)</code> — the queue routing key that extends single-writer-per-table across the roster.
Events	<code>src/quarry/daemon/fs_events.py</code>	<code>FsEvent</code> + <code>FsEventSource</code> protocol seam (tests inject synthetic events).
Watchdog	<code>src/quarry/daemon/fs_watchdog.py</code>	The sole <code>watchdog</code> importer: <code>FSEvents</code> (macOS) / <code>inotify</code> (Linux) with a <code>PollingObserver</code> fallback (on <code>ENOSPC</code>).
Debounce	<code>src/quarry/daemon/debounce.py</code>	Per-path debounce window + max-delay cap + bulk-threshold coalescing.
Index jobs	<code>src/quarry/daemon/index_jobs.py</code>	<code>FileIndexJob</code> / <code>DocumentDeleteJob</code> / <code>CollectionSyncJob</code> (<code>IngestUnits</code>).
Finalize	<code>src/quarry/daemon/finalize_job.py</code>	<code>CollectionFinalizeJob</code> (<code>SyncFinalizer</code>): the lone post-quiescence <code>optimize()</code> + FTS rebuild, coalesced by <code>FinalizeThrottle</code> (DES-045c).

The loop feeds the same DES-042 serialized queue (`src/quarry/daemon/ingest_queue.py`, `src/quarry/daemon/coll`) one resident FIFO worker per `RouteKey` (one in-flight `progressive_insert` per LanceDB table) behind a global embed `Semaphore` hard-clamped to 1 (DES-032: more than one buys no matmul parallelism behind the shared ONNX session mutex and re-adds arena contention). Admission is non-blocking and bounded — a full queue returns 503 and the event re-arms with capped backoff, never a silent drop.

A small delta re-indexes as per-file jobs (embed gate released between files for cross-collection fairness); a burst above `watch_bulk_threshold` (default 50) collapses to one `CollectionSyncJob` (DES-034's size-gated flush keeps the fragment budget). Per-file jobs *never* rebuild the FTS index — a lone finalize job runs post-quiescence per `RouteKey`, so the daemon's persistent connection does not reopen the quarry-0dss descriptor leak (`create_fts_index(replace=True)` pins deleted-file readers the LanceDB Rust core never evicts); a resource-invariant test proves the fd count plateaus across multiple databases.

Removal lifecycle and orphan sweep (DES-045a). Deregister, subsumption, and orphan cleanup were hardened under a Z model (`docs/spec/watch_lifecycle.tex`, fuzz-clean and ProB-checked against ten invariants I1–I10). A collection is marked in a durable `pending_purge_collections`

table — in the *same* transaction as the `directories` delete — only on a non-keep-data deregister or a subsume eviction; the periodic reconcile’s orphan sweep then purges only *marked* collections that still hold chunks, re-checking under its own transaction at execution time. Non-directory collections (captures, `memory-*`, the `default remember` target, URL-host collections) are never marked, so the sweep is *structurally unable* to delete them (I9) — the invariant whose earlier absence, in a model that wrongly assumed every collection was a directory registration, would have wiped all captures and agent memories on the 5-minute default timer. A `--keep-data` disable archives the collection’s identity (`retained_collections` with `original_directory`); only the *same* directory may re-adopt it (I7), and re-adoption triggers a reconcile that prunes chunks for files deleted while it was disabled. A different directory always gets a fresh, distinct collection, and the fresh-name picker avoids every chunk-bearing name so the auto path is merge-proof (I10).

Watch scope (DES-045b). The `ScratchGuard` predicate, consulted at every producer seam (initial scan, live watch, reconcile, explicit `quarry sync`), refuses a root that is (a) an OS temp root (`/tmp`, `/private/tmp`, `/var/tmp`, `/private/var/tmp`, `/var/folders`, compared casefolded so a case variant on APFS cannot slip past) or (b) a repo’s own scratch, anchored on `<gitroot>/tmp` for *any* ancestor git repo; below a permitted root it skips gitignored paths and an always-skip set (`.git`, `.beads`, `.venv`, `node_modules`, `__pycache__`, `dist`, `build`, `cache`s). Resolution is fail-closed *per root* — a `resolve()` that raises skips that one registration and logs, never aborting the scan. The guard fires at *watch* time, not only at register time, so an already-registered temp root is refused on daemon restart with no registry edit. This closes the hole where a stale `/private/tmp` registration made the loop re-index the churning system temp dir and never reach quiescence (283 CPU-minutes observed before the guard shipped).

CPU/compaction envelope (DES-045c). DES-045’s “bounded by construction” bounded `fds`, memory, and the write-serializer, but LanceDB sizes its own tokio *compute* runtime to `num_cpus`, and a single `finalize` (`optimize()` + native FTS rebuild) on that uncapped pool seized every core (291–403% CPU observed). Two structural mitigations bound it: (1) `ThreadConfig.apply_env_limits()` (`src/quarry/thread.config.py`) clamps `LANCE_CPU_THREADS/LANCE_IO_THREADS` (cap 2) before the first `lancedb.connect` in `DaemonServer.run()`, via a fail-closed clamp with a per-variable floor (a stale `LANCE_CPU_THREADS=32` is forced down to the cap, and `=1` — which stalls lance’s runtime — is raised to the floor of 2, so floor equals ceiling for the LANCE pools); (2) `FinalizeThrottle` coalesces the `finalize` to at most one per `watch_optimize_min_interval_s` (default 30 s) *per database*, with a trailing timer so FTS still catches up after churn settles. The vector channel is never stale (files index immediately); the reconcile is the unthrottled backstop.

3.10 Surfaces

Module	File	Responsibility
CLI	src/quarry/_main_.py + src/quarry/cli_*.py	Typer CLI with rich formatting. A thin <code>QuarryClient</code> client: build request → call daemon → format output. Command groups split into <code>src/quarry/cli_search</code> , <code>src/quarry/cli_ingest</code> , <code>src/quarry/cli_documents</code> , <code>src/quarry/cli_sync</code> , <code>src/quarry/cli_project</code> , <code>src/quarry/cli_remote</code> , <code>src/quarry/cli_maintenance</code> .
MCP Server	src/quarry/mcp_server.py	<code>quarry mcp</code> : a FastMCP <i>stdio</i> server whose eleven tool bodies are thin <code>QuarryClient</code> calls over the daemon's <code>/v1</code> API. Imports <i>no</i> engine — loads zero LanceDB/ONNX (DES-031 v2.2).
Client tier	src/quarry/client/	<code>QuarryClient</code> + injectable REST transport, typed <code>QuarryError</code> hierarchy, <code>ClientConfig</code> target/token resolution. Replaces the deleted <code>RemoteClient</code> .
Wire API	src/quarry/api/	The <code>/v1</code> contract: one Pydantic request/response model per operation + a uniform <code>ErrorBody</code> . Drift is an import-time type error (the structural kill of bug class 3).
Hooks	src/quarry/hooks.py	Claude Code plugin hooks: session-start (auto-register), post-web-fetch (auto-ingest), pre-compact (capture transcript). Pure clients (<code>QuarryClient</code>); all fail-open.
Hook Entry	src/quarry/_hook_entry.py	Lightweight hook dispatcher that bypasses full CLI import chain. Dispatches via <code>sys.argv</code> with lazy imports (~0.1s vs ~1.5s for full CLI).
Remote	src/quarry/remote.py	Remote login config: TOFU CA cert fetch and storage, pinned-CA connection validation.
Daemon	src/quarry/service.py	<code>quarryd</code> lifecycle via <code>launchd/systemd</code> .

3.11 Support

Module	File	Responsibility
Formatting	src/quarry/formatting.py	Pre-formatted plain text output using unicode box-drawing. Used by both CLI and MCP for consistent output.
Sitemap	src/quarry/sitemap.py	Sitemap discovery and URL extraction for web ingestion.
Doctor	src/quarry/doctor.py	Health checks and install flow: model availability, database access, MCP configuration, ethos extension setup.
Enable	src/quarry/enable.py	Project activation and deactivation: register directory, bootstrap ethos memory extensions, write per-project config.
TLS	src/quarry/tls.py	TLS certificate generation: self-signed EC P-256 CA and server cert with atomic file writes and TOFU-safe CA reuse.
Proxy	src/quarry/proxy.py	<code>mcp-proxy</code> binary download, SHA256 verification, and platform-specific installation.
Logging	src/quarry/logging_config.py	Structured logging configuration: rotating file handler + stderr, <code>QUARRY_LOG_LEVEL</code> override.

4 Daemon Model

4.1 Why a Daemon

The embedding model (snowflake-arctic-embed-m-v1.5, int8 quantized ONNX) occupies ~200MB of RAM and takes ~250ms to load. MCP's stdio transport model spawns a new process per session. Without a daemon, five Claude Code tabs means five copies of the model in memory and five cold starts.

`quarryd` loads the model once and keeps it resident, respawned by `launchd KeepAlive / systemd Restart=always`.

4.2 Process Boundary and Loopback Authentication (v2.2)

`quarryd` (`src/quarry/daemon/launcher.py`) is a dedicated entry point — the sole engine process, making the client/engine boundary a hard *process* split. Only `src/quarry/daemon/` imports the engine; the `quarry serve` subcommand is deleted.

At startup `quarryd` refuses a remote-reachable bind that carries no operator key (an auto-generated token would be false security — it is unreadable by the remote clients that need it), and for a loopback bind mints a 256-bit `serve.token` when none is supplied. The daemon writes that token mode-0600 beside `serve.port` (shared `src/quarry/run_dir.py`) via an atomic `os.open(0o600) + tmp-rename`, and requires it on every request — closing the multi-user-host exposure where any local UID could reach the unauthenticated daemon on 127.0.0.1. Loopback classification (`src/quarry/net.py`, `LoopbackPolicy`) is `ipaddress`-based: `localhost/::1/127.0.0.0/8` are loopback, `0.0.0.0` and unresolved names fail closed.

Clients resolve the target through `ClientConfig` (`src/quarry/client/config.py`): for a loopback URL the bearer is read *live* from `serve.token` (it rotates every daemon restart, so a stored token is never trusted); for a remote URL the stored bearer is kept. A missing loopback token fails closed with an actionable error rather than a bare 401. `ClientConfig` feeds the one `QuarryClient` that the CLI, the `quarry mcp` stdio server, hooks, and the library all use; `RemoteClient` is deleted.

The daemon serves the **REST API only** under `/v1` (the in-daemon MCP route was removed in PR-1); its request-admission trust model combines the `JsonContentTypeGuard` — an ASGI CSRF choke point that rejects any POST without `Content-Type: application/json` (a browser cannot send that cross-origin without a preflight the CORS policy refuses) — with the `serve.token` bearer above. The guard blocks a forged cross-origin write; the token blocks an unauthorized local reader.

4.3 Client/Engine Boundary (I1)

The daemon-first split (DES-031) is a hard boundary, enforced *structurally* rather than by convention: a client process never imports the engine. Three layers, dependency arrow pointing inward: `src/quarry/quarry/api` (the wire contract — Pydantic models, zero engine, zero heavy deps) ← `src/quarry/quarry/client` (`QuarryClient` transport + typed errors, imports only `api`) ← the client processes (`src/quarry/_main_`, `src/quarry/hooks`, `src/quarry/mcp.server`). The engine — `src/quarry/db/`, `src/quarry/embeddings`, `src/quarry/ingestion/`, `src/quarry/retrieval/`, `src/quarry/sync`, `src/quarry/daemon/` — is server-side only; its entry point `src/quarry/daemon/launcher.py` (the `quarryd` script) lives inside `src/quarry/daemon/`, so it is engine-side and never a client.

Two mechanisms lock it (PR-6):

- **An import-linter contract** (`.importlinter`, run by `make check-imports` in `make check` and CI) forbids any of the five client modules from importing any of the six engine packages along a *transitive* chain; a violating import fails CI, not review. The one sanctioned exception is an explicit, enumerated `ignore_imports` list of the host-admin diagnostic edges (`doctor/install/uninstall` probe the local engine environment to report why a daemon is unhealthy) — function-body lazy imports off the hot path, not an invisibility trick (grimp resolves lazy imports).
- **A runtime engine-sabotage test** imports each client module in a subprocess with `lancedb/onnxruntime/pyarrow` poisoned and asserts no engine module reaches *module* scope — the companion guard that the diagnostic lazy imports stay deferred.

Daemon-dependent tests run against a hermetic in-process ASGI fixture (`httpx.ASGITransport` over the FastAPI `build_app`, no socket); a single real-loopback-TLS smoke (slow tier, excluded from the fast suite) covers the pinned-CA wire contract end-to-end; and `install` gates on `/health ready` before exiting 0.

4.4 MCP Transport (quarry mcp)

Quarry’s plugin runs `quarry mcp` directly — a FastMCP *stdio* server (`src/quarry/mcp_server.py`) whose eleven tool bodies are thin `QuarryClient` calls over the daemon’s `/v1` REST API. It reads MCP JSON-RPC from `stdin`, dispatches each tool to the resident daemon (reusing its one warm `ctx.embedder/ctx.database`), and relays the response to `stdout`. It imports no engine, so `import quarry.mcp_server` and running `quarry mcp` load zero LanceDB/ONNX. Database selection is a per-request parameter carried in the wire call; the daemon holds no per-connection `ContextVar` because the in-daemon `/mcp` WebSocket route was removed (DES-031 v2.2, PR-1). `mcp-proxy` is no longer on this path (it remains a supported MCP `stdio↔daemon` transport for other MCP clients).

4.5 CLI and the Daemon

The CLI is a thin client: it builds a request, calls the daemon over the `/v1` wire protocol via `QuarryClient`, and renders the response — it does not load the embedding model or open LanceDB in-process. The prior design had `quarry find "margins"` load the model in-process and query LanceDB directly; that local-engine path is removed. The one sanctioned exception is the host-admin diagnostics (`doctor/install/uninstall`), which probe the local engine environment through function-body lazy imports so they can report *why* a daemon is unhealthy even when it is down — an explicit, enumerated `ignore_imports` list, proven off the hot path by the sabotage test.

5 MCP Tool Surface

Tool	Key Parameters	Purpose	Exec
<code>find</code>	<code>query</code> , <code>limit</code> , <code>collection</code>	Semantic search	Sync
<code>ingest</code>	<code>source</code> , <code>overwrite</code> , <code>collection</code>	Ingest file or URL	Bg
<code>remember</code>	<code>content</code> , <code>document_name</code>	Ingest inline text	Bg
<code>list</code>	<code>kind</code>	List indexed data	Sync
<code>show</code>	<code>document_name</code> , <code>page_number</code>	Document metadata or page	Sync
<code>delete</code>	<code>name</code> , <code>kind</code>	Delete document or collection	Bg
<code>register_directory</code>	<code>directory</code> , <code>collection</code>	Register for sync	Bg
<code>deregister_directory</code>	<code>collection</code> , <code>keep_data</code>	Remove registration	Bg
<code>sync_all</code>	—	Sync all registered dirs	Bg
<code>status</code>	—	Database stats	Sync
<code>use</code>	<code>name</code>	Switch active database	Sync

Bg = background (daemon-side; the tool returns a `task_id`).

Each tool body is a thin `QuarryClient` call over `/v1` (the MCP server holds no engine). A background tool posts to the daemon, which returns 202 `Accepted` with a `task_id`; the actual work runs on the daemon’s serialized per-`RouteKey` queue (DES-042) behind the embed `Semaphore`, and the caller polls `/v1/tasks/{task_id}` or ignores the outcome. This keeps a 30-second ingest from blocking the LLM response stream while preventing the concurrent-write races the single-writer-per-table invariant closes. Read tools return their result inline.

6 HTTP Server

6.1 ASGI Application

FastAPI app built by `build_app()` (`src/quarry/daemon/app.py`), served by `uvicorn`. FastAPI *is* Starlette underneath, so request validation *and* the OpenAPI document derive from the same `src/quarry/quarry/api` Pydantic models — no hand-rolled emitter or route registry. REST handlers are sync functions run in

the threadpool; the routes live in `src/quarry/daemon/routes/`. There is no WebSocket endpoint — the daemon serves the **REST API only** (the in-daemon `/mcp` route was removed in DES-031 v2.2, PR-1).

Default port: 8420 (`DEFAULT_PORT` in `src/quarry/config.py`), overridable with `--port`. All engine operations sit under `/v1`; `/health` and `/ca.crt` are unversioned. `/health` advertises `api_version`; a major mismatch raises `QuarryVersionError` in the client.

6.2 Endpoints

Path	Method	Purpose	Auth	Model
<code>/health</code>	GET	Server state + <code>api_version</code>	No	sync
<code>/ca.crt</code>	GET	CA certificate PEM (TOFU)	No	sync
<code>/v1/search</code>	GET	Hybrid search	Yes	sync
<code>/v1/show</code>	GET	Document metadata or page text	Yes	sync
<code>/v1/documents</code>	GET	List documents	Yes	sync
<code>/v1/documents</code>	DELETE	Delete document (returns 202)	Yes	async
<code>/v1/collections</code>	GET	List collections	Yes	sync
<code>/v1/collections</code>	DELETE	Delete collection (returns 202)	Yes	async
<code>/v1/registrations</code>	GET	List directory registrations	Yes	sync
<code>/v1/registrations</code>	POST	Register directory (returns 202)	Yes	async
<code>/v1/registrations</code>	DELETE	Deregister directory (sync + 202 purge)	Yes	async
<code>/v1/databases</code>	GET	List named databases	Yes	sync
<code>/v1/status</code>	GET	Database statistics	Yes	sync
<code>/v1/use</code>	POST	Switch active database	Yes	sync
<code>/v1/remember</code>	POST	Ingest inline content (returns 202)	Yes	async
<code>/v1/ingest</code>	POST	Ingest file or URL (returns 202)	Yes	async
<code>/v1/capture</code>	POST	Capture web/transcript (returns 202)	Yes	async
<code>/v1/captures/push</code>	POST	Push private capture shadow repo	Yes	async
<code>/v1/sync</code>	POST	Sync registrations (returns 202)	Yes	async
<code>/v1/optimize</code>	POST	Compact + rebuild index (returns 202)	Yes	async
<code>/v1/backfill-sessions</code>	POST	Backfill session transcripts (returns 202)	Yes	async
<code>/v1/tasks/{task_id}</code>	GET	Poll task status (unified)	Yes	sync

`/v1/tasks/{task_id}` is the single, unified poll endpoint — the old `/sync/{id}` and `/ingest/{id}` alias routes were removed in the DES-031 v2.2 contract rewrite. See Appendix A for the full concurrency matrix across HTTP, CLI, and MCP surfaces.

6.3 Port File

After binding, the daemon writes the actual port to `~/.punt-labs/quarry/data/<db>/serve.port`, and (when it has an effective key) a mode-0600 `serve.token` beside it (see §4.2). This supports `--port 0` (ephemeral assignment). Removed on shutdown via the lifespan context.

7 Configuration

7.1 Settings

All settings are environment variables read via `pydantic-settings`.

Variable	Default	Description
Paths		
<code>QUARRY_ROOT</code>	<code>~/.punt-labs/quarry/data</code>	Base directory for all databases
<code>LANCEDB_PATH</code>	<code><root>/default/lancedb</code>	Vector database location

Variable	Default	Description
REGISTRY_PATH	<root>/default/registry.db	Directory sync SQLite database
LOG_PATH	~/punt-labs/quarry/logs/quarry.log	Log file (rotating, 5 MB, 3 backups; independent of QUARRY_ROOT)
Chunking		
CHUNK_MAX_CHARS	1800	Target max characters per chunk (~4 kens)
CHUNK_OVERLAP_CHARS	200	Overlap between consecutive chunks
Embedding		
EMBEDDING_MODEL	Snowflake/snowflake-arctic-embed-m-v1.5	Model identifier (display only — the model is fixed)
EMBEDDING_DIMENSION	768	Vector dimension (display only — fixed)
Progressive commit (DES-034)		
SYNC_FLUSH_MB	32	Flush buffered vectors to LanceDB once reach this many MB (bounds peak re vectors; may fire mid-file)
EMBED_WINDOW_CHUNKS	512	Chunks embedded per window, so a largument never materializes all its vectors throughput seam)
Watch/index loop (DES-045)		
WATCH_ENABLED	true	Run the always-on watch/index loop; stalling quarryd is the opt-in; revers prior timer posture)
WATCH_DEBOUNCE_S	1.0	Coalesce an edit burst before submitting
WATCH_MAX_DELAY_S	5.0	Cap on a continuously-rearmed path starvation)
WATCH_BULK_THRESHOLD	50	A delta above this many paths collapses bulk CollectionSyncJob
WATCH_USE_POLLING	false	Force watchdog's stat-walk PollingObs
WATCH_SAFETY_SCAN_S	300.0	Periodic roster reconcile + orphan picks up new databases and re-submit scans (0 = off)
WATCH_OPTIMIZE_MIN_INTERVAL_S	30.0	FinalizeThrottle: at most one finalize + FTS rebuild) per interval per da (0 = every batch)
Thread limits (process env; DES-032/045c)		
LANCE_CPU_THREADS / LANCE_IO_THREADS	2	Clamp LanceDB's tokio compute/IO (fail-closed, floor == cap == 2; set before first lancedb.connect)
OMP_NUM_THREADS	min(2, ncpu)	Cap OpenMP to match the ONNX in limit (floor 1)

The LanceDB and OMP caps are applied by `ThreadConfig` (`src/quarry/thread_config.py`) in `DaemonServer.run()` via a fail-closed clamp: an inherited or operator value is honored only within `[floor, cap]`; a higher or non-numeric value is forced *down* to the cap (a `setdefault` would yield upward to a stale export and void the ceiling) and a below-floor value is raised to the floor. ONNX `intra_op_num_threads` is `min(2, ncpu)` on CPU and 1 on GPU; `inter_op_num_threads` is 1 (DES-032).

7.2 Named Databases

`resolve_db_paths(settings, name)` maps a database name to a path under `QUARRY_ROOT`. Each database is fully isolated — its own LanceDB directory, sync registry, and vector index. Names containing path separators (`/` or `\`) or the literal traversal segments `.` or `..` are rejected; names like `foo.bar` are allowed. If `LANCEDB_PATH` was explicitly overridden, the override is preserved.

The persistent default database is stored in `~/.punt-labs/quarry/config.toml` as `[default] database = "name"`.

7.3 Chunking Tuning

Increase `CHUNK_MAX_CHARS` (3000–4000) for documents with long self-contained sections: legal contracts, academic papers, large source code functions.

Decrease `CHUNK_MAX_CHARS` (800–1200) for dense documents where each paragraph covers a distinct topic.

Increase `CHUNK_OVERLAP_CHARS` (400) when relevant information spans paragraph boundaries.

The embedding model has a 512-token context window. Chunks exceeding this are truncated during embedding, reducing quality for the tail.

8 Search and Retrieval

8.1 Embedding Model

snowflake-arctic-embed-m-v1.5 via ONNX Runtime:

- 768-dimensional vectors, 512-token context window
- Asymmetric retrieval: queries are prefixed with a search instruction, documents are not
- Output vectors are L2-normalized to unit length, so search runs under the **cosine** metric and similarity ($1 - \text{distance}$) is bounded true cosine in $[-1, 1]$ (DES-038)
- int8 quantized ONNX for CPU (~120 MB), FP16 for CUDA (~220 MB)
- Provider auto-detection: CUDA+FP16 when available, CPU+int8 fallback. `QUARRY_PROVIDER` env var overrides (`cpu`, `cuda`, or `unset` for auto). See DES-016 and `src/quarry/ingestion/provider.py`.
- Auto-downloads on first use via [huggingface.hub](#)

The model is fixed — there is no configuration to swap it. Changing the model invalidates all existing embeddings, requiring full re-ingestion.

8.2 OCR

RapidOCR handles all image-based text extraction. For semantic search, OCR accuracy matters less than expected — the embedding model is robust to minor OCR errors. A misspelled word rarely changes semantic meaning enough to affect search ranking.

8.3 Search Quality Tips

1. **Use natural language queries.** The model is trained on question-passage pairs. “What were Q3 revenue figures?” works better than “Q3 revenue.”
2. **Use collection filtering.** `quarry find "auth logic" -c backend`.
3. **Re-ingest after tuning.** Chunk parameters only affect new ingestions. Run `quarry sync` after changes.
4. **Check chunk boundaries.** `quarry find -n 1 "query"` and examine the result.
5. **Use `--overwrite` when re-ingesting.** Without it, duplicate chunks accumulate.

8.4 Hybrid Search

Hybrid search is the **default for every query** — there is no vector-only path. All three surfaces (CLI, HTTP, MCP) construct a `SearchService` (`src/quarry/retrieval/service.py`) and call its one `search` method, so the surfaces cannot drift into different retrieval behavior. This closes bug class 3 (remote/local divergence), where the HTTP `/search` endpoint once ran vector-only search while the CLI ran hybrid. `SearchService` wraps a `HybridRetriever` parameterized by a `RetrievalConfig` whose defaults reproduce the shipped pipeline bit-for-bit.

Channels:

1. **Vector similarity** — ANN search via LanceDB under the cosine metric
2. **BM25 full-text search** — LanceDB’s *native* (Rust) inverted index on the `text` column, the FTS backend LanceDB builds by default, created during schema migration. Quarry’s `LanceTable` protocol and the `RecyclingTable` wrapper forward only `replace` to `create_fts_index`; there is no separate Tantivy `IndexWriter` process or thread pool, so the index rides the daemon’s capped LanceDB runtime (DES-045c). (Earlier ADRs — DES-017, DES-023 — name the backend “Tantivy”; that name is legacy, the shipped engine is LanceDB-native.)

Results are fused using Reciprocal Rank Fusion (RRF):

$$\text{score}(m) = \sum_{\text{channel}} \frac{1}{60 + \text{rank}_{\text{channel}}(m)}$$

FTS scoring. FTS rows arrive from the full-text channel without a vector distance. Rather than displaying a synthetic 1.00 (which made irrelevant keyword hits appear as perfect matches), each FTS-only row is annotated with its *true* cosine distance against the query vector; a row whose stored vector is missing or zero-length is assigned the worst-case distance (similarity -1) instead.

Temporal decay is not a second retrieval channel — it is an additional weighting term applied only to agent-scoped memories (non-empty `agent_handle`). An `agent_handle` filter therefore narrows the result set and enables decay; it does not switch retrieval modes.

$$w = e^{-\lambda \cdot h}$$

where λ is the decay rate and h is hours since ingestion. Unscoped documents (`agent_handle = ""`) are exempt from decay (production leaves $\lambda = 0$).

8.5 Retrieval Seam and Evaluation

The `src/quarry/retrieval/` package is a deliberate seam (DES-037): the same `HybridRetriever` that serves production also serves the retrieval evaluation harness (`make eval`), so the committed baseline measures the retriever that actually ships. `RetrievalConfig` exposes the levers a bake-off varies — RRF k , over-fetch multiplier, distance metric, exact-vs-ANN vector search, reranker, and embedding strategy — without forking the code path. The harness is internal dev/CI tooling; it is not an end-user command. See `docs/eval-harness-design.md`.

Three columns extend the chunks table for agent-scoped memory:

Column	Type	Default	Purpose
<code>agent_handle</code>	utf8	""	Agent identity scope
<code>memory_type</code>	utf8	""	fact, observation, opinion, procedure
<code>summary</code>	utf8	""	One-line summary

Migration is idempotent — `_migrate_schema()` adds missing columns with empty-string defaults on every table open. Existing databases gain the columns transparently.

Identity tagging: the PreCompact hook reads the agent handle from `.punt-labs/ethos/config.yaml` and tags captured transcripts automatically.

8.6 Ethos Extension Integration

Quarry stores its configuration in ethos’s generic extension mechanism (DES-008, DES-019):

```
~/.punt-labs/ethos/identities/<handle>.ext/quarry.yaml
```


The extension file contains two categories of data:

1. **Sidecar config** — `memory_collection` and `expertise_collections` are read directly by quarry via the filesystem (no ethos API call).
2. **Session context** — `session_context` is a YAML literal block scalar emitted verbatim by ethos at session start and before context compaction. It contains the agent’s memory instructions (collection name, slash commands, memory types).

`quarry install` (step 7/7) scans all identity extension directories and appends `session_context` to any `quarry.yaml` that has `memory_collection` but no `session_context`. Uses raw file append to preserve existing YAML comments and formatting. Per-identity exception handling ensures one malformed file does not abort the rest.

Ownership boundary: ethos emits quarry’s instructions without understanding them. Quarry writes its own instructions without importing ethos. The dependency is one-way — quarry depends on ethos for identity; ethos has zero knowledge of quarry’s internals.

9 Logging and Exceptions

9.1 Logging Standard

Every module that performs I/O declares `logger = logging.getLogger(__name__)`. Pure data modules (`models.py`, `types.py`, `chunker.py`) do not.

Logging is configured once at the application entry point (CLI or MCP server). Library modules never call `logging.basicConfig`.

Level	Use
DEBUG	Internal state: variable values, branch decisions, intermediate results.
INFO	Operational milestones: starting a job, completing a step, resource counts.
WARNING	Unexpected but recoverable: empty input, deprecated parameter usage.
ERROR	Failures that prevent completion but do not crash. Always with exception context.

CRITICAL is not used. If the process must exit, it raises an exception.

Log messages use `%s`-style formatting (lazy evaluation), not f-strings. Each message includes enough context to trace an operation without reading code.

9.2 Exception Handling

The codebase uses built-in exception types. No custom exceptions unless a caller needs to distinguish failure modes programmatically.

Exception	Raised when
<code>ValueError</code>	Invalid input: unsupported format, bad parameter.
<code>FileNotFoundError</code>	A file path argument does not exist.
<code>RuntimeError</code>	An operation reports failure (OCR engine error, ingestion failure).
<code>TimeoutError</code>	An external service exceeds its time budget.

Rules:

1. Never swallow exceptions. Every `except` block must re-raise, or log with `logger.exception()` and raise a different exception, or (at system boundaries only) return a documented sentinel.
2. Never use bare `except:`.

3. Never catch `Exception` broadly in library code. Broad catches are permitted only at the outermost boundary (MCP tool handler, CLI command).
4. Exceptions carry context: include the values that caused the failure.
5. Use `try/finally` for cleanup, not `try/except`.
6. Do not use exceptions for flow control.

10 Security

10.1 Bearer Token Authentication

`quarryd --api-key` (or `QUARRY_API_KEY` env var) gates all HTTP endpoints except `/health` behind `Authorization: Bearer <key>`.

- `/health` is exempt — load balancers need unauthenticated health checks.
- `OPTIONS` is exempt — CORS preflight requests carry no auth.
- `hmac.compare_digest` for token comparison — prevents timing attacks.
- Case-insensitive scheme per RFC 7235.
- Empty key = no auth — prevents accidental trivial bypass.
- No key = auth disabled — local use requires no key.

The server refuses to bind to a non-loopback address without `--api-key`.

10.2 Request Admission (CSRF)

The daemon serves REST only, so there is no WebSocket to hijack; the cross-origin threat is a forged browser `POST`. Two checks compose the admission trust model:

- `JsonContentTypeGuard` — an ASGI choke point that rejects any `POST` without `Content-Type: application/json`. A browser cannot send that header cross-origin without a preflight the CORS policy refuses, so a forged cross-origin write is blocked at the door.
- The `serve.token` / API-key bearer (checked on every `/v1` request) blocks an unauthorized *local* reader.

10.3 CORS

`CORSMiddleware` configured with GET and OPTIONS methods, Authorization and Content-Type headers. Default allowed origin: `http://localhost`. Configurable via `--cors-origin` (repeatable).

The server reflects the request's `Origin` header only when it matches the allow list, and adds `Vary: Origin` to signal cacheability varies by origin. When no `Origin` is present, no CORS headers are emitted.

10.4 TLS and TOFU Certificate Pinning

The installed service always runs with TLS enabled. `quarryd` without `--tls` is supported for local development but must not be used for production. Certificate management uses TOFU (Trust On First Use) with self-signed CA pinning (DES-020).

Certificate generation: `quarry install` generates an EC P-256 CA and server certificate. Files are written atomically to `~/punt-labs/quarry/tls/` with restricted permissions (0600 for keys, 0644 for CA cert). The server cert SAN includes the configured hostname (via `QUARRY_TLS_HOSTNAME`), `localhost`, and loopback addresses.

TOFU login:

1. Client fetches `/ca.crt` over HTTPS (verification disabled — bootstrap).
2. Displays SHA256 fingerprint for out-of-band confirmation.

3. Pins CA cert at `~/punt-labs/mcp-proxy/quarry-ca.crt`.
4. All subsequent connections verify against the pinned CA only — system roots excluded.

Pinned-CA consumers: `QuarryClient` builds its TLS context from the pinned CA as the sole root (system roots excluded), so every daemon RPC — CLI, `quarry mcp`, hooks, library — verifies against it. The `quarry.toml` profile also records the CA under `ca_cert`, which `mcp-proxy` reads (as the sole root, TLS 1.3 minimum) when it transports MCP for other MCP clients; it is no longer on quarry's own MCP path (DES-031 v2.2).

10.5 Log Safety (CWE-532, CWE-117)

Uvicorn's access log is disabled (`access.log=False`), eliminating query string leakage at the source. The search handler logs only result count, never the raw query. (The WebSocket session-key sanitization the prior design described was removed with the in-daemon `/mcp` route — there is no longer a WebSocket surface to log.)

10.6 Capture PII Redaction (DES-036)

Captures — session transcripts and WebFetch auto-captures — are the input to the planned private shadow-repo sync (`quarry-ow3k`), so un-redacted content in a capture is a leak into a git-tracked (and soon pushed) surface. Every capture is scrubbed **at write time**, fail-closed, before any bytes reach disk.

Scrubber (`src/quarry/scrub.py`). A **Scrubber** applies redaction passes in a load-bearing order: *secrets* → *paths* → *emails* → *hostname* → *profanity*. Email must precede hostname so a hostname inside an email domain is subsumed by the whole-email redaction rather than half-redacted (which would leak the local part). Each pass emits a marker no pass can re-match, so scrubbing is idempotent — backfill may safely re-run.

- **Secrets** (`src/quarry/scrub_rules.py`): GitHub PATs, AWS keys, Anthropic/OpenAI keys, bearer headers, JWTs, PEM/GPG private key blocks, `KEY=value` env secrets, Slack tokens.
- **Paths**: home directories (`/Users/<user>/`, `/home/<user>/`) collapse to `~`, retaining the useful deeper structure.
- **Emails**: any RFC-shaped address → `[REDACTED:email]`. A placeholder redacts every address (completeness beats attribution), not an ethos-handle mapping.
- **Hostname**: bounded to the local machine name (`socket.gethostname() + .local` + short leaf), not arbitrary dotted-token detection — the latter has a catastrophic false-positive rate on identifiers like `github.com` or `quarry.db.facade`.

Single write choke point (`src/quarry/capture.py`). Both `.md` producers — the `PreCompact` hook (`src/quarry/hooks.py`) and the backfill path (`src/quarry/backfill.py`) — route through one `CaptureWriter`. Scrubbing runs to completion before an atomic temp-file rename, so a scrub failure writes no file and a partial or half-redacted capture is never left behind.

WebFetch DB-ingest. The WebFetch auto-capture path scrubs page text via an opt-in `content_scrubber` parameter on `ingest_url` (`src/quarry/ingestion/pipeline.py`). The parameter defaults to `None`: user-initiated `quarry ingest <url>`, sitemap crawls, and directory sync are *not* scrubbed — redaction is a captures concern, not a general-ingestion one. The URL's persisted metadata form is redacted separately by `CaptureUrl` (`src/quarry/capture_url.py`), which strips `userinfo`, `query`, and `fragment` (preserving IPv6 brackets) before the same text scrubber runs over the bare `scheme://host/path`.

11 Deployment

11.1 Single Machine

```
curl -fsSL https://raw.githubusercontent.com/.../install.sh | sh
```

Installs quarry, embedding model, mcp-proxy, Claude Code plugin, and registers a daemon with TLS on localhost.

11.2 Network Mode (Remote GPU)

```
export QUARRY_API_KEY=$(openssl rand -hex 32)
curl -fsSL https://raw.githubusercontent.com/.../install.sh | sh -s -- --network
```

Same as default, but binds daemon to 0.0.0.0:8420 instead of localhost. Requires `QUARRY_API_KEY`. Installs quarry, embedding model, generates TLS certificates, registers systemd service. Detects NVIDIA GPUs and swaps `onnxruntime` for `onnxruntime-gpu`. Prints CA fingerprint for client TOFU. Claude Code plugin installed if `claude` CLI is on PATH.

11.3 Client (Remote Access)

```
curl -fsSL https://raw.githubusercontent.com/.../install.sh | sh
quarry login <server> --api-key <token>
```

No special flag needed. Default install runs a local daemon on localhost. `quarry login` redirects queries to the remote server via TOFU certificate pinning and writes the mcp-proxy config.

11.4 Daemon (quarryd)

```
QUARRY_API_KEY=<token> quarryd --port 8420 --host 0.0.0.0 --tls
```

Non-loopback hosts require an API key (the server refuses to start without one).

Managed by `launchd` (macOS) or `systemd` (Linux) via `quarry install`. API key delivered via `systemd EnvironmentFile` (Linux) or `launchd EnvironmentVariables` (macOS) — never in process arguments.

11.5 Container (Fly.io)

Multi-stage Dockerfile, three stages:

1. **deps** (`python:3.13-slim`): installs `uv`, runs `uv sync --frozen --no-dev`. Two-pass for Docker layer caching (deps first, then source).
2. **model** (extends **deps**): downloads the embedding model at build time (~120 MB, int8 quantized). Cached as a separate Docker layer.
3. **runtime** (`python:3.13-slim`): copies `/app` from **deps**, copies model cache from **model** stage. `QUARRY_ROOT=/data`, port 8080.

LanceDB data lives on a Fly persistent volume at `/data`. `QUARRY_API_KEY` is set via `fly secrets`. Cold start is ~5s with the baked-in model.

12 Performance

12.1 Critical Path

Operation	Latency	Notes
Model load	~250 ms	One-time per process
Embedding (1 chunk)	~50 ms	Apple Silicon, CPU
LanceDB search	<10 ms	Vector similarity + filter
End-to-end search	<100 ms	After model is loaded

12.2 Fire-and-Forget Writes

Side-effect operations return immediately with a `task_id`; the work runs on the daemon’s serialized per-`RouteKey` queue (DES-042). This is necessary because:

1. MCP tool calls block the LLM’s response stream. A 30-second ingest would block Claude from responding. The 202 unblocks the caller immediately.
2. Bounded, non-blocking admission (503 on a full queue) prevents the resource exhaustion that unbounded `asyncio.create_task` ingests caused (the load-90 starvation, quarry-lnog).
3. One in-flight `progressive_insert` per LanceDB table (single writer per `RouteKey`) plus the embedded `Semaphore` clamped to 1 eliminate the concurrent delete-then-add lost-update race, without snapshotting mutable per-request state.

12.3 Single Table Design

All chunks live in one LanceDB table (`chunks`). Document and collection boundaries are columns, not separate tables. This simplifies cross-document search and avoids table proliferation. Filtering by `document/collection/page_type/source_format` uses LanceDB’s built-in filter predicates on the vector search.

13 Plugin and Hook System

The Claude Code plugin (`.claude-plugin/`) registers hooks that fire on session events and tool calls. All hooks are fail-open: exceptions are logged and the process exits 0.

Hook Event	Matcher	Effect	Blocks?
SessionStart	*	Auto-register directory + background sync	No
PostToolUse	quarry tools	Suppress tool output formatting	No
PostToolUse	WebFetch	Auto-ingest fetched URL (scrubbed)	No
PreCompact	*	Capture transcript as session notes (scrubbed)	No

13.1 Session Start

`handle_session_start()` auto-registers the working directory and launches a background sync via a detached subprocess. Uses atomic PID-file locking (`O_CREAT|O_EXCL`) at `~/.punt-labs/quarry/sync.pid` to prevent concurrent syncs. Returns `additionalContext` immediately without waiting.

13.2 Web Fetch Capture

`handle_post_web_fetch()` ingests URLs fetched during a session into the `web-captures` collection. Deduplicates by document name. Prefers already-fetched content from the tool response (avoids re-fetch, reduces SSRF surface). Both ingress branches (primary and the `ingest_url` re-fetch fallback) scrub page text and redact the URL metadata before content reaches the pushable `web-captures` collection (DES-036).

13.3 Pre-Compact Capture

`handle_pre_compact()` reads the transcript (JSONL) and ingests it into the `session-notes` collection. Extracts user/assistant message text, skipping tool-use blocks. Capped at 500,000 characters. The capture file is written through the shared `CaptureWriter`, which scrubs secrets, PII, and profanity before the atomic write (DES-036).

14 CLI Commands

Command	Purpose
<code>ingest <source></code>	Ingest file or URL. Flags: <code>--agent-handle</code> , <code>--memory-type</code> , <code>--summary</code>
<code>remember</code>	Ingest stdin content
<code>find <query></code>	Semantic search
<code>show <document></code>	Document metadata or page text
<code>status</code>	Database statistics
<code>use <name></code>	Set persistent default database
<code>delete <name></code>	Delete document or collection
<code>register <directory></code>	Register for incremental sync
<code>deregister <collection></code>	Remove directory registration
<code>sync</code>	Sync all registered directories
<code>enable [directory]</code>	Enable quarry knowledge capture for a project directory
<code>disable [directory]</code>	Disable quarry knowledge capture; remove registration and data
<code>optimize</code>	Compact LanceDB table and rebuild indexes (CLI-only). Flags: <code>--force</code>
<code>list documents</code>	List indexed documents
<code>list collections</code>	List collections with counts
<code>list databases</code>	List named databases
<code>list registrations</code>	List registered directories
<code>serve</code>	Start HTTP API server
<code>mcp</code>	Start MCP server (stdio)
<code>login <host></code>	TOFU login to remote server (cert pinning)
<code>logout</code>	Disconnect from remote server
<code>remote list</code>	Show remote config; <code>--ping</code> validates connection
<code>install</code>	Set up data directory, download model, generate TLS certs, register daemon, detect GPU, configure ethos ext
<code>doctor</code>	Check environment health
<code>uninstall</code>	Remove system daemon
<code>version</code>	Print version

Global flags: `--json`, `--verbose/-v`, `--quiet/-q`, `--db <name>`.

Remote routing (DES-021): When a remote server is configured (via `quarry login`), all data commands should route to the remote server over HTTPS with the pinned CA cert. Only authentication commands (`login`, `logout`, `remote`) and administration commands (`install`, `uninstall`, `serve`, `mcp`, `version`) are local-only. All data commands — `find`, `show`, `status`, `ingest`, `remember`, `delete`, `sync`, every `list` subcommand, and directory `register/deregister` — route remotely. `use` is local-only by design (the remote server is fixed to its own database).

15 Test Architecture

Roughly 1,500 tests across the suite. Three tiers plus integration:

Tier	Strategy	Files
Tier 1: Core logic	Real data, no mocks	test_database, test_chunker, test_embeddings, test_code_processor, test_html_processor, test_text_processor, test_collections, test_config, test_formatting, test_image_analyzer, test_latex_utils, test_models, test_pdf_analyzer, test_presentation_processor, test_spreadsheet_processor, test_text_extractor, test_registry (17 files)
Tier 2: Orchestration	Mocked I/O	test_pipeline, test_pipeline_images, test_ocr_local, test_url_ingestion, test_sitemap, test_backends, test_sync, test_hooks, test_doctor (9 files)
Tier 3: Surface wiring	Fully mocked / hermetic in-process ASGI (httpx.ASGITransport), <2s	test_cli, test_mcp_server, test_http_server, test_client_config, test_daemon_launcher, test_watch_loop, test_watch_reconcile, test_fs_watchdog, test_proxy, test_service
Integration	Real ONNX + real LanceDB	test_integration (@pytest.mark.slow)

Quality gates: **ruff** (lint + format), **mypy** (strict), **pyright** (strict), **pytest** (all markers except slow/integration). Zero violations required.

16 Formal Model

The plugin state machine — how quarry hooks into the Claude Code session lifecycle — is formally specified in `docs/claude-code-quarry.tex` using Z notation (ISO 13568).

The specification covers:

- Session start with/without auto-sync
- Web fetch auto-capture into **web-captures** collection
- Pre-compaction transcript capture into **session-notes**
- Database switching via **use** tool
- Knowledge hint injection at tool boundaries
- Compaction observability: a capture flag records whether capture preceded compaction; the hook system enforces capture-before-compaction on a best-effort basis
- Clean session boundaries: all quarry state cleared on **EndSession**

The specification can be type-checked with `fuzz -t claude-code-quarry.tex`.

A Operation Concurrency Model

Every quarry operation is exposed on up to three surfaces: the HTTP API (`/v1`, daemon), the CLI, and the MCP server (`quarry mcp studio`). The concurrency model differs by surface because each has different constraints: HTTP connections time out, MCP tool calls block the LLM response stream, and CLI commands run in a user’s terminal. Under daemon-first every CLI and MCP call is a `QuarryClient` request to the daemon, so the “local” and “remote” CLI columns below differ only in transport target, not in code path.

Table 4 classifies each operation. The **Client model** column describes what the caller experiences:

sync	The caller blocks until the operation completes and receives the result inline.
fire-and-forget	The caller receives an acknowledgement immediately (HTTP 202 or a short string). Work runs in the background; the caller polls or ignores the outcome.
—	The operation is not available on that surface.

A.1 Concurrency Matrix

Table 4: Operation concurrency by surface. *Long-running* = involves network I/O, embedding, or bulk LanceDB writes.

Operation	HTTP API	CLI (local)	CLI (remote)	MCP
<i>Read operations</i>				
find (search)	sync	sync	sync	sync
show	sync	sync	sync	sync
status	sync	sync	sync	sync
list documents	sync	sync	sync	sync
list collections	sync	sync	sync	sync
list registrations	sync	sync	sync	sync
list databases	sync	sync	sync	sync
health	sync	—	—	—
<i>Mutating operations</i>				
register	fire-and-forget	sync	fire-and-forget	fire-and-forget
deregister	fire-and-forget	sync	fire-and-forget	fire-and-forget
delete (document)	fire-and-forget	sync	fire-and-forget	fire-and-forget
delete (collection)	fire-and-forget	sync	fire-and-forget	fire-and-forget
remember	fire-and-forget	sync	fire-and-forget	fire-and-forget
ingest (URL)	fire-and-forget	sync	fire-and-forget	fire-and-forget
ingest (file)	—	sync	—	fire-and-forget
sync	fire-and-forget	sync	fire-and-forget	fire-and-forget
use (switch db)	sync	sync	sync	sync
optimize	fire-and-forget	sync	fire-and-forget	—
backfill-sessions	fire-and-forget	sync	fire-and-forget	—

A.2 Design Rationale

Read operations are synchronous on all surfaces. Hybrid search (embedding + vector similarity + BM25) completes in <100 ms after model load; the overhead of task tracking would exceed the operation itself.

Mutating operations are all fire-and-forget on HTTP. Every mutating endpoint returns 202 Accepted with a `task_id`; work runs in a background `asyncio.Task`. This eliminates the 15-second CLI timeout as a failure mode for any write operation.

- All mutating endpoints use the single unified GET `/v1/tasks/{task_id}` for status polling; the old `/sync/{id}` and `/ingest/{id}` alias routes were removed in the DES-031 v2.2 contract rewrite.

- **Sync** no longer returns 409: it enqueues transparently (202 + poll), because the daemon’s serialized per-RouteKey queue (DES-042) already prevents concurrent writers to a table. The 409 is retained only for **optimize** and **backfill-sessions**, whose whole-table compaction/rebuild must not overlap itself.
- Completed and failed tasks are garbage-collected after a 1-hour TTL on the next task creation.
- **Optimize** and **backfill-sessions** are exposed on HTTP and the CLI but *not* as MCP tools — they are whole-database maintenance, run with explicit intent rather than from an agent’s tool loop.

A.3 Task Lifecycle

All background tasks follow the same lifecycle:

1. POST validates input, creates a **TaskState** record on the server context, spawns an **asyncio.Task** wrapping **run_in_threadpool()**, and returns HTTP 202 with **task_id** and **status = accepted**.
2. The background task updates **status** to **completed** (with **results**) or **failed** (with **error**) on completion.
3. A **finally** guard marks any task that exits without setting a terminal status as **failed**.
4. **CancelledError** is caught before **Exception** to ensure clean shutdown on server stop.
5. The CLI prints the **task_id** and exits 0 (fire-and-forget).